

Guide d'integration des donnees reelles depuis PlatformIO (ESP32) vers XAMPP

1. Principe general

Le systeme IoT est compose d'un microcontrôleur ESP32 et d'un capteur a ultrasons (ex : HC-SR04) place sous le couvercle du bac. Ce capteur mesure la distance restante entre le haut du bac et les dechets. L'ESP32 calcule le pourcentage de remplissage et l'envoie en HTTP POST sous format JSON a la plateforme web BENINCLEAN. L'application traite les donnees en temps reel dans la base de donnees XAMPP.

2. Route API de Reception Laravel

J'ai configure l'application Laravel pour recevoir les donnees IoT sur l'URL suivante :
POST http://<IP_DU_PC>:8000/api/bins/telemetry

Cette route est exemptee de la validation de jeton CSRF (via bootstrap/app.php) pour permettre aux objets connectes d'y acceder directement. Elle valide que le code du bac existe en base de donnees et met a jour son taux de remplissage.

Format JSON attendu par l'API :

```
{
  "code": "B105",
  "fill_level": 75
}
```

Exemple de script de traitement dans BinController.php :

```
public function updateTelemetry(Request $request)
{
    $validated = $request->validate([
        "code" => "required|string|exists:bins,code",
        "fill_level" => "required|integer|min:0|max:100"
    ]);
    $bin = Bin::where("code", $validated["code"])->firstOrFail();
    $bin->fill_level = $validated["fill_level"];
    // recalcul automatique du statut (Normal, Presque Plein, Plein) et gestion des alertes...
    $bin->save();
}
```

3. Code source C++ pour l'ESP32 sur PlatformIO

Voici le code source C++ a implementer sur votre projet PlatformIO. Il utilise la bibliotheque ArduinoJson pour formater les donnees et HTTPClient pour envoyer la requete HTTP POST :

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

const char* ssid = "VOTRE_NOM_WIFI";
const char* password = "VOTRE_MOT_DE_PASSE_WIFI";
const char* serverUrl = "http://192.168.1.50:8000/api/bins/telemetry"; // Remplacer par l'IP de votre PC
```

```
const int trigPin = 12; // Broche Trig du capteur HC-SR04
const int echoPin = 13; // Broche Echo du capteur HC-SR04
const float depthOfBin = 100.0; // Profondeur totale du bac en cm

void setup() {
    Serial.begin(115200);
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500); Serial.print(".");
    }
    Serial.println("\nWiFi connecte !");
}

void loop() {
    // Mesure de distance par ultrasons
    digitalWrite(trigPin, LOW); delayMicroseconds(2);
    digitalWrite(trigPin, HIGH); delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    long duration = pulseIn(echoPin, HIGH);
    float distance = duration * 0.034 / 2.0;
    if (distance > depthOfBin) distance = depthOfBin;

    // Calcul du pourcentage de remplissage
    int fillLevel = (int)((depthOfBin - distance) / depthOfBin) * 100;
    if (fillLevel < 0) fillLevel = 0;

    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        http.begin(serverUrl);
        http.addHeader("Content-Type", "application/json");

        StaticJsonDocument<200> doc;
        doc["code"] = "B102"; // Mettre le code correspondant au bac
        doc["fill_level"] = fillLevel;

        String requestBody;
        serializeJson(doc, requestBody);
        int httpStatusCode = http.POST(requestBody);
        if (httpStatusCode > 0) {
            Serial.println("Donnees envoyees ! Reponse : " + http.getString());
        } else {
            Serial.println("Erreur d'envoi : " + String(httpStatusCode));
        }
        http.end();
    }
    delay(15000); // Envoi toutes les 15 secondes pour le test
}
```

4. Configuration Réseau requise (Important)

1. L'ESP32 et le PC executant XAMPP doivent etre connectes sur le MEME reseau Wi-Fi (ex: votre telephone en partage de

INTEGRATION IOT PLATFORMIO

BENINCLEAN - Interface Web de Monitoring & Gestion des Bacs

connexion ou la box internet du domicile).

2. Ne pas utiliser 'localhost' or '127.0.0.1' dans le code de l'ESP32 car pour l'ESP32, localhost designe lui-meme. Vous devez utiliser l'adresse IP locale de votre ordinateur (ex: 192.168.1.50).
3. Pour trouver l'IP de votre PC, ouvrez le terminal PowerShell et tapez 'ipconfig'. Cherchez 'Adresse IPv4'.
4. Lancez le serveur Laravel avec la commande suivante pour qu'il ecoute sur tout le reseau et pas seulement sur localhost :
`php artisan serve --host=0.0.0.0 --port=8000`